# DC MOTOR PI CONTROL: AUTOMATIC OPTIMISATION

1.0 Learning Outcomes.....1
2.0 Hardware Set-Up.....1
3.0 Practical Investigation via Simulink.....1
4.0 Concluding Remarks.....7
Recommended Citation.....7

## 1.0 Learning Outcomes

Upon completing this practical exercise, students should be able to demonstrate the Learning Outcomes presented in Table 1. At the end of the exercise, revisit each Learning Outcome and confirm that it has been achieved.

**Table 1:** *Learning Outcomes for the Practical Exercise*

| ID | Learning Outcome | Achieved |
|---|---|---|
| 1 | Build a simulation of the closed-loop speed control system from the transfer function identified by linear least squares (LLS), including saturation, anti-windup, encoder quantisation and filtering. | ☐ |
| 2 | Write the engineering requirements into a cost function, and use `fminsearch` to optimise the PI gains so that the requirements are met for motor gains between ×0.8 and ×1.2. | ☐ |
| 3 | Implement the optimised controller on the hardware, verify it against the engineering requirements, and explain any differences from the simulation. | ☐ |

## 2.0 Hardware Set-Up

This practical exercise builds on the hardware and algorithm set-up described in the practical exercises, DC Motor Speed Control Using a PI Controller, and Parameter Estimation of a DC Motor using LLS, which detail closed-loop speed control of the DC motor and the identification of its transfer function. The set-up is not repeated here.

The Simulink model from DC Motor Speed Control Using a PI Controller is used unchanged: the encoder on pins 2 and 3 is sampled every 0.01 s, the measured speed passes through a Discrete Filter, and a discrete-time PI controller, with clamping anti-windup, drives the Cytron Maker Drive through pin 5. The motor is supplied from a 9 V battery, and the controller output is limited to a PWM command of 0 to 170 so that no more than 6 V reaches the motor.

## 3.0 Practical Investigation via Simulink

MATLAB and Simulink should be used to complete this practical exercise. In the practical exercise, DC Motor PI Control: Manual Tuning, the gains were found by trial and error on the hardware. Here, the trial and error is carried out by a

computer, in simulation, using the transfer function identified in Parameter Estimation of a DC Motor using LLS. The optimiser, `fminsearch`, adjusts the gains, simulates the closed loop, and keeps the changes that reduce a cost calculated from the requirements, as described in the PI Controller Tuning by Numerical Optimisation Learning Guide.

The controller is judged against the engineering requirements in Table 2. Requirements R1 to R5 are checked on the hardware; R6 is checked in simulation, since the gain of the motor cannot easily be changed on the hardware.

**Table 2:** *Engineering Requirements*

| ID | Engineering requirement |
|---|---|
| R1 | The DC motor shall track a reference speed of 30 RPM. |
| R2 | The steady-state error, averaged over the last 2 s of the step, shall be no more than 0.5 RPM. |
| R3 | The 10–90% rise time shall be less than 1.0 s. |
| R4 | The overshoot shall be no more than 5%. |
| R5 | The controller output shall remain within the permitted actuator range of 0–170. |
| R6 | R3 to R5 shall also be met, in simulation, when the gain of the motor is changed by ±20%. |

## The Identified Model of the Motor

In Parameter Estimation of a DC Motor using LLS, a first-order model was identified from a step test. The example ACE-Lab motor gave:

$$G(s) = \frac{187}{s + 6.67} = \frac{28}{0.15s + 1} \ \mathrm{RPM/V} \tag{1}$$

that is, a gain of $K \approx 28$ RPM/V and a time constant of $\tau \approx 0.15$ s. In that exercise, the logged input was the PWM command multiplied by 5/255, so the same scaling is used here: the PWM command is multiplied by 5/255 before it enters $G(s)$, and the model is used exactly as it was identified.

1. Record the transfer function of your own motor, from Table 3 of Parameter Estimation of a DC Motor using LLS, in Table 3 below. Use the model identified without the low-pass filter, since the filter is simulated separately.
2. Record the coefficients of the Discrete Filter in your Simulink model. If it is a first-order filter, $H(z) = (1-a_f)/(1-a_f z^{-1})$, calculate its time constant from $\tau_f = -T_s/\ln a_f$. The example uses $\tau_f = 0.1$ s.

**Table 3:** *Identified Model and Filter of Your Motor*

| | Identified $G(s)$ | $K$ (RPM/V) | $\tau$ (s) | Filter time constant $\tau_f$ (s) |
|---|---|---|---|---|
| Example ACE-Lab motor | 187/ ($s$+6.67) | 28 | 0.15 | 0.1 |
| Your motor | | | | |

### Optimise the PI Gains

The script below tunes the PI gains automatically. The function `piCost` simulates the closed loop exactly as it runs on the Arduino: the zero-order-hold model of Equation (1), the encoder with 3576 counts per revolution of the output shaft, the low-pass filter, the PI controller with a trapezoidal integrator and clamping anti-windup, and the PWM command limited to 0–170 and rounded to an integer. It returns a cost made up of the ITAE index and penalty terms for each requirement, as in Equation (3) of the learning guide. The targets include a margin: a rise time of 0.6 s against the 1.0 s requirement, and an overshoot of 4% against the 5% requirement.

For R6, every cost evaluation simulates three motors, with gains of ×0.8, ×1.0 and ×1.2, and the optimiser minimises the worst of the three costs. The search starts from $K_p$ = 2 and $K_i$ = 5; you may instead start from the gains you selected in DC Motor PI Control: Manual Tuning.

**MATLAB Script:** optimising the PI gains on the identified model (pi_opt_tune.m)

```
clear; clc; close all;

%% Model, controller and requirements
P.num = 187;  P.den = 6.67;     % your model: G(s) = num/(s + den)
P.tauf = 0.1;                   % your filter time constant [s]
P.Ts = 0.01;  P.T = 4;          % sample time, step length [s]
P.ref = 30;  P.umax = 170;      % reference [RPM], PWM limit
P.CPR = 3576;                   % encoder counts per output rev
P.tr_target = 0.6;              % rise time target [s] (R3, margin)
P.os_target = 4;                % overshoot target [%] (R4, margin)
scales = [0.8 1.0 1.2];         % motor gain variation (R6)

%% Robust optimisation with fminsearch
x0 = log([2 5]);                % starting gains: Kp, Ki
J = @(x) max(arrayfun(@(s) piCost(exp(x), P, s), scales));
opts = optimset('Display', 'iter', 'TolX', 1e-3, 'TolFun', 1e-5, ...
                'MaxFunEvals', 300);
xb = fminsearch(J, x0, opts);
Kp = exp(xb(1));  Ki = exp(xb(2));
fprintf('\nKp = %.3f PWM/RPM, Ki = %.2f PWM/(RPM s)\n', Kp, Ki);

%% Predicted performance for each motor
figure; hold on; grid on;
for s = scales
    [~, M, S] = piCost([Kp Ki], P, s);
    fprintf('Gain x%.1f: rise %.2f s, OS %.1f %%, SSE %.3f RPM, ', ...
            s, M.tr, M.os, M.sse);
    fprintf('max PWM %d\n', max(S.u));
    plot(S.t, S.yf);
end
yline(P.ref, 'k--');
xlabel('Time (s)'); ylabel('Filtered speed (RPM)');
legend(compose('Motor gain x%.1f', scales), 'Location', 'southeast');

function [J, M, S] = piCost(g, P, sc)
Kp = g(1);  Ki = g(2);  Ts = P.Ts;  n = round(P.T/Ts);
a = exp(-P.den*Ts);                     % ZOH model of G(s)
b = P.num/P.den*(1 - a)*5/255;          % input: PWM x 5/255
af = exp(-Ts/P.tauf);                   % low-pass filter pole
y = 0;  pos = 0;  cPrev = 0;  yf = 0;  I = 0;  ePrev = 0;
t = (0:n-1)'*Ts;  yfL = zeros(n,1);  uL = zeros(n,1);
for k = 1:n
    pos = pos + y*Ts/60*P.CPR;  c = floor(pos);   % encoder count
    ym = (c - cPrev)*60/(P.CPR*Ts);  cPrev = c;   % measured speed
    yf = af*yf + (1 - af)*ym;                     % filtered speed
    e = P.ref - yf;  v = Kp*e + I;                % PI, parallel form
    if ~((v > P.umax && e > 0) || (v < 0 && e < 0))
        I = I + Ki*Ts*(e + ePrev)/2;              % clamping anti-windup
    end
    ePrev = e;
    u = round(min(max(v, 0), P.umax));            % PWM command 0-170
```

```
    yfL(k) = yf;  uL(k) = u;
    y = a*y + sc*b*u;                              % motor model
end
% performance measures
i10 = find(yfL >= 0.1*P.ref, 1);  i90 = find(yfL >= 0.9*P.ref, 1);
if isempty(i10) || isempty(i90)
    tr = P.T + (0.9*P.ref - max(yfL))/P.ref*P.T;  % 90 % not reached
else
    tr = t(i90) - t(i10);
end
os = max(0, (max(yfL) - P.ref)/P.ref*100);
ss = t > P.T - 2;  sse = mean(P.ref - yfL(ss));
act = mean(abs(diff(uL(ss))));                     % actuator activity
itae = sum(t.*abs(P.ref - yfL))*Ts/(P.ref*P.T^2/2);
J = itae + 0.5*act/P.umax + 20*max(0, tr/P.tr_target - 1)^2 ...
    + 5*max(0, os/P.os_target - 1)^2 + 20*(sse/0.5)^2;
M = struct('tr', tr, 'os', os, 'sse', sse);
S = struct('t', t, 'yf', yfL, 'u', uL);
end
```

1. Enter the transfer function and filter time constant of your motor, from Table 3, at the top of the script.
2. Run the script. Watch the iterations displayed by `fminsearch`, and note how many cost evaluations are needed.
3. Record the optimised gains and the predicted performance for each motor in Table 4. Does the predicted performance meet R3, R4 and R5 for all three motors (R6)?
4. Change the starting point to $K_p$ = 1, $K_i$ = 1, and run the script again. Explain the result, using Section 4.0 of the PI Controller Tuning by Numerical Optimisation Learning Guide. Then try at least one other starting point, for example $K_p$ = 5, $K_i$ = 20, and record whether the same gains are found.

   The tuning process is shown in Figure 1. For the example motor, starting from $K_p$ = 2 and $K_i$ = 5, `fminsearch` converged after about 90 cost evaluations to $K_p$ = 2.14 PWM/RPM and $K_i$ = 10.1 PWM/(RPM s). The predicted responses are shown in Figure 2 and summarised in Table 4. The starting gains reach the reference only slowly, with a rise time of 0.88 s; the optimised gains rise in 0.21 to 0.33 s for all three motors, with no more than 4% overshoot, and use no more than 78 of the 170 PWM counts available. Your results may differ slightly, depending on your model and on the version of MATLAB.

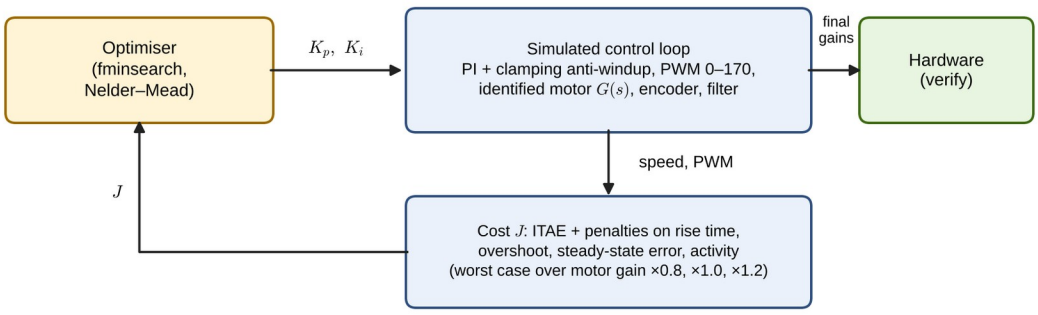



**Figure 1:** *Automatic Optimisation of the PI Gains: the Optimiser Proposes Gains, the Closed Loop Is Simulated with the Identified Model, and the Cost Is Returned*

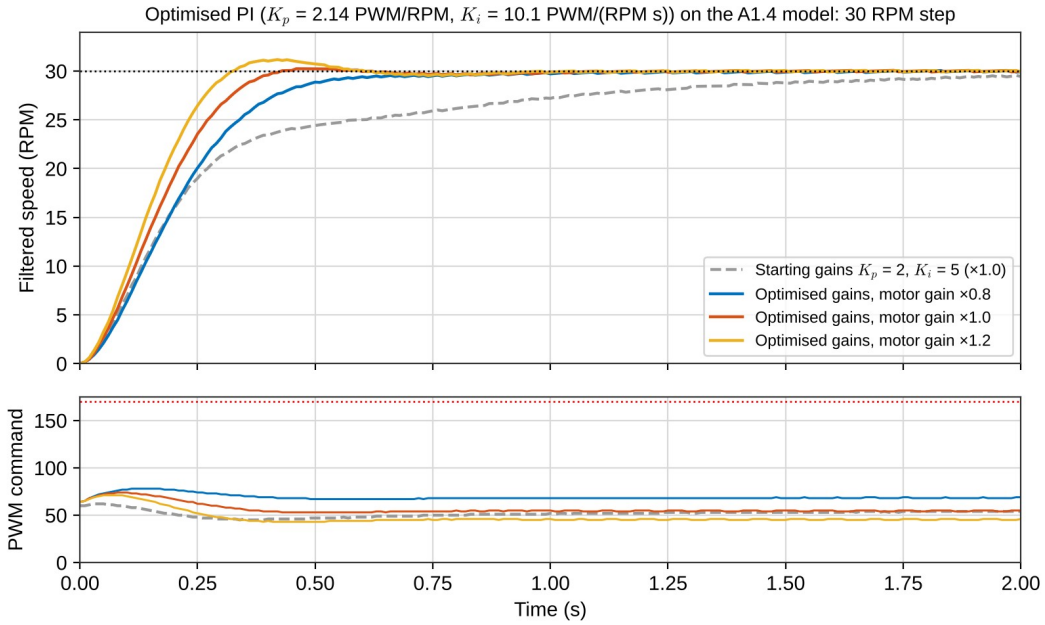



**Figure 2:** *Predicted 30 RPM Step Responses of the Optimised PI Controller for Motor Gains of ×0.8, ×1.0 and ×1.2, Compared with the Starting Gains (Simulation)*

**Table 4:** *Optimised Gains and Predicted Performance*

| | Motor gain | $K_p$ (PWM/RPM) | $K_i$ (PWM/(RPM s)) | Rise time (s) | Overshoot (%) | Steady-state error (RPM) | Peak PWM command |
|---|---|---|---|---|---|---|---|
| Example motor | ×0.8 | 2.14 | 10.1 | 0.33 | 0.2 | 0.00 | 78 |
| | ×1.0 | 2.14 | 10.1 | 0.26 | 0.8 | 0.00 | 74 |
| | ×1.2 | 2.14 | 10.1 | 0.21 | 3.9 | 0.00 | 71 |
| Your motor | ×0.8 | | | | | | |
| | ×1.0 | | | | | | |
| | ×1.2 | | | | | | |

### Implement and Verify the Controller on the Hardware

The optimised gains are now used in the Simulink model from DC Motor Speed Control Using a PI Controller. The Pulse Generator applies a 30 RPM reference for 5 s, and the speed then returns to zero.

1. Open your Simulink model. In the PID Controller block, set the Proportional gain to $K_p$ and the Integral gain to $K_i$ from Table 4. Check that the controller is in discrete time with a sample time of 0.01 s, that the integrator uses the trapezoidal method, and that the output is limited to 0–170 with clamping anti-windup, as assumed in `piCost`.
2. Log the reference speed, the filtered speed and the controller output to the workspace, using the Scope logging settings from Parameter Estimation of a DC Motor using LLS (Structure With Time), with the variable names `r`, `yf` and `u`.
3. Run the model on the hardware for 10 s.
4. Run the script below to measure the response against R1 to R5.

**MATLAB Script:** measuring the hardware response against the requirements

```
t  = yf.time;  y = yf.signals.values(:);  u_k = u.signals.values(:);
on = t >= 0 & t < 5;                       % the 30 RPM step
S  = stepinfo(y(on), t(on), 30);
last = t >= 3 & t < 5;                     % last 2 s of the step
sse  = mean(30 - y(last));
fprintf('Rise time %.2f s (R3 < 1 s)\n', S.RiseTime);
fprintf('Overshoot %.1f %% (R4 <= 5 %%)\n', S.Overshoot);
fprintf('Steady-state error %.2f RPM (R2 <= 0.5 RPM)\n', sse);
fprintf('PWM command %d to %d (R5: 0-170)\n', min(u_k), max(u_k));
```

5. Record your results in Table 5, and state whether each requirement is met.

**Table 5:** *Engineering Requirements Compared Against the Measured Response*

| ID | Engineering requirement | Predicted (simulation) | Measured result | Requirement met? |
|---|---|---|---|---|
| R1 | Track a reference speed of 30 RPM | | _____ RPM | Yes/No |
| R2 | Steady-state error no more than 0.5 RPM | | _____ RPM | Yes/No |
| R3 | Rise time less than 1.0 s | | _____ s | Yes/No |
| R4 | Overshoot no more than 5% | | _____ % | Yes/No |
| R5 | Controller output within 0–170 | | _____ | Yes/No |
| R6 | R3 to R5 met for motor gains of ×0.8 and ×1.2 | | (simulation) | Yes/No |

Example results are shown in Figure 3. **These results are from a simulation of the example motor, not from the hardware.** To represent the hardware more closely, a dead-band of 0.2 V, which the identified model does not contain, was added to the simulated motor. The response rose in 0.30 s with 0.3% overshoot and no measurable steady-state error, and the PWM command peaked at 77, so R1 to R5 were met. Note how the unfiltered speed jumps between neighbouring values: one encoder count in a 0.01 s sample is about 1.68 RPM.

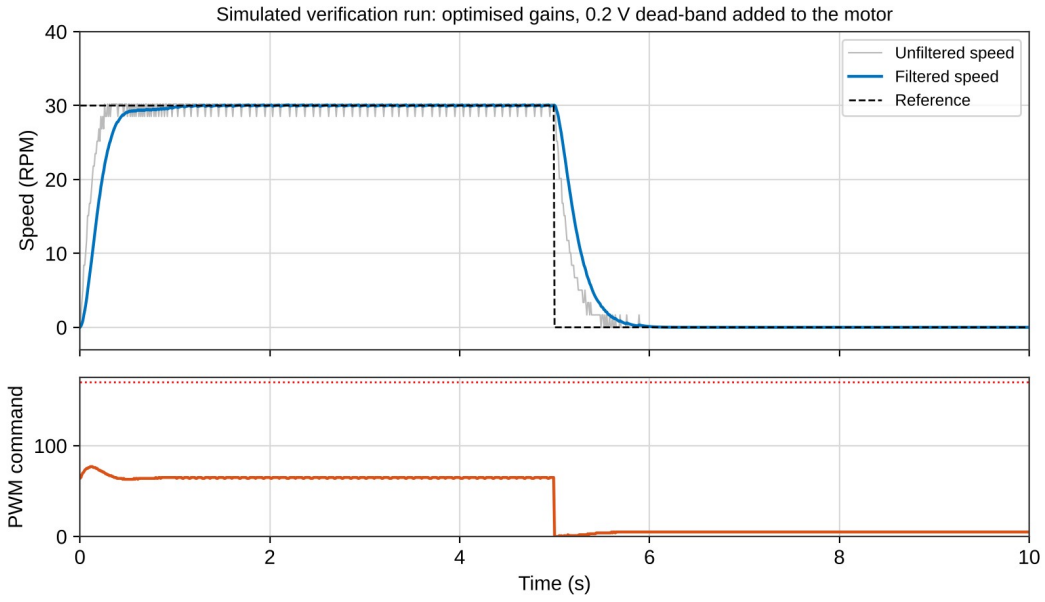



**Figure 3:** *Example Verification Run of the Optimised PI Controller (Simulation of the Example Motor with a 0.2 V Dead-Band): Speed (Top) and PWM Command (Bottom)*

Compare your measured and predicted performance, and explain any differences. In your explanation, consider the following:

- The model contains no dead-band and no friction. How would these affect the response near zero speed, and the PWM command needed to hold 30 RPM?
- Was your model identified with the same battery voltage and load as the verification run? How large a change in the gain of the motor does R6 allow for?
- The optimiser was told to aim for a rise time of 0.6 s and an overshoot of 4%. What would happen on the hardware if the targets had been set exactly at the requirements, 1.0 s and 5%?
- Which starting points led to the best gains, and how would you check that the gains you used are the best that the method can find?

## 4.0 Concluding Remarks

This practical exercise tuned a PI speed controller for the ACE-Lab DC motor automatically. Using MATLAB, Simulink and ACE-Lab hardware, the transfer function identified by linear least squares was used to build a simulation of the closed loop that included saturation, anti-windup, encoder quantisation and filtering. The engineering requirements were written into a cost function, and `fminsearch` adjusted the gains until the cost was minimised for motor gains between ×0.8 and ×1.2. The optimised controller was then verified on the hardware against the requirements.

This practical exercise also showed that the result of an optimiser is only as good as the problem it is given. The cost function decides which requirements are traded against each other, the margins in the targets allow for differences between the model and the hardware, and the starting point decides which local minimum is found. Every step, from the measured data to the gains, can be repeated and checked, and the same method can be applied to any model that can be simulated.

## Recommended Citation

If you have used this practical exercise in your teaching, learning or research, please cite it using the reference below, adapting the format as required by your preferred referencing style.

Pickering, J. E. (2026). DC Motor PI Control: Automatic Optimisation. ACE-Lab. Teaching Notes: ACE-Lab Practical Exercises, Version 1.0. Zenodo. https://doi.org/10.5281/zenodo.23675491